

ENST2



M2DS - École Polytechnique

APM 5DS24 TP - Deep Learning II
Probabilistic Generative Models

Final Project - Deep Belief Network
and Generative Modeling

By:

Clément Destouesse

Ralph Mouawad

To:

Prof. Yohan Petetin

Prof. Alasdair Newson

Contents

1	Abstract	3
2	Probabilistic Generative Models	4
2.1	Restricted Boltzmann Machines (RBM)	4
2.1.1	Definition	4
2.1.2	Gibbs Sampling	5
2.1.3	Contrastive Divergence 1 (CD-1)	6
2.2	Deep Belief Network (DBN)	7
2.2.1	DBN as a Stack of RBMs	7
2.2.2	Greedy Layer-Wise Procedure	7
2.3	Deep Neural Networks	7
3	Algorithmic Implementation and Results	8
3.1	Image Generation through RBM and DBN	8
3.2	Comparison DNN : pre-trained vs random	12
3.3	Comparison DNN : layers, neurons, data	14
4	Advanced Generative Models on MNIST	17
4.1	RBMs	17
4.2	DBN	18
4.3	Variational Autoencoders (VAEs)	18
4.4	Generative Adversarial Networks (GANs)	19
4.5	Denoising Diffusion Probabilistic Models (DDPM)	20
4.6	Score-Based Generative Modeling (SGM)	22
4.7	Comparison and Conclusion	23
5	Code	24

List of Figures

1	Graphical Representation of a RBM	5
2	Training curves for the RBM and DBN models.	9
3	Generated samples obtained with the RBM for different numbers of characters in the training set and different hidden-layer sizes.	10
4	Generated samples obtained with the DBN for different numbers of characters in the training set and different architectures.	11
5	Sample for comparison between pre trained and random DNN	13
6	Heatmap probability for models output	14
7	Evolution of the error depending on the number of layers	15
8	Evolution of the error depending on the number of neurons	16
9	Evolution of the error depending on the number of training data	16
10	Digits generated with RBMs	17
11	Digits generated with DBN	18
12	Digits generated with diagonal covariance matrix	19
13	Digits generated with non-diagonal covariance matrix	19
14	Digits generated with VAE	19
15	Digits generated with GAN	20
16	Digits generated with DDPM	22
17	Digits generated with SGM	23

1 Abstract

This project focuses on the study of generative deep learning models and their ability to learn meaningful representations of handwritten characters and digits. In the first part, we investigate unsupervised generative models based on Restricted Boltzmann Machines (RBMs) and Deep Belief Networks (DBNs), with the objective of analyzing their capacity to reconstruct and generate realistic images. A central goal is to evaluate the quality of image generation under different settings, including the number of hidden units, the number of hidden layers in the DBN architecture, and the number of character classes used during training. In the second part, we study the supervised learning setting through the training of a Deep Neural Network (DNN) for classification. More specifically, we compare standard supervised training with an approach based on unsupervised pretraining using stacked RBMs, followed by supervised fine-tuning. We then analyze the impact of architectural and training choices on performance. As a bonus extension, we also explore more recent generative approaches, including Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), as well as modern methods based on diffusion models and flow matching, in order to compare their image generation capabilities with those of RBMs and DBNs.

2 Probabilistic Generative Models

Probabilistic generative modeling aims at learning a probability distribution that explains how the observed data are produced. Once this distribution is estimated, the model can generate new samples that follow the same statistical patterns as the training set. Such models are useful not only for data generation, but also for density estimation, representation learning, and the completion or reconstruction of partially observed data.

2.1 Restricted Boltzmann Machines (RBM)

2.1.1 Definition

A Restricted Boltzmann Machine is an energy-based probabilistic model with two layers of binary units: a visible layer v , which encodes the observed data, and a hidden layer h , which captures latent features. If the visible layer contains p units and the hidden layer contains q units, then

$$\begin{aligned} v_i &\in \{0, 1\}, \quad i = 1, \dots, p, \\ h_j &\in \{0, 1\}, \quad j = 1, \dots, q. \end{aligned}$$

The model assigns a probability to each pair (v, h) through an energy function $E(v, h)$:

$$p(v, h) = \frac{1}{Z} e^{-E(v, h)},$$

where Z is the partition function defined by

$$Z = \sum_{v, h} e^{-E(v, h)}.$$

For a binary RBM, the energy is written as

$$E(v, h) = - \sum_{i=1}^p a_i v_i - \sum_{j=1}^q b_j h_j - \sum_{i=1}^p \sum_{j=1}^q w_{ij} v_i h_j,$$

where a_i and b_j denote the biases of the visible and hidden units, and w_{ij} measures the interaction between visible unit i and hidden unit j . The parameter set of the RBM is therefore

$$\theta = (a, b, W).$$

A key structural property of the RBM is that connections only exist across the two

layers: there are no visible-visible or hidden-hidden connections. Because of this bipartite architecture, the hidden units become independent when the visible layer is fixed, and similarly the visible units are independent when the hidden layer is fixed. This leads to simple conditional probabilities:

$$p(h_j = 1 | v) = \sigma \left(b_j + \sum_{i=1}^p w_{ij} v_i \right),$$

$$p(v_i = 1 | h) = \sigma \left(a_i + \sum_{j=1}^q w_{ij} h_j \right),$$

where

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

is the sigmoid function.

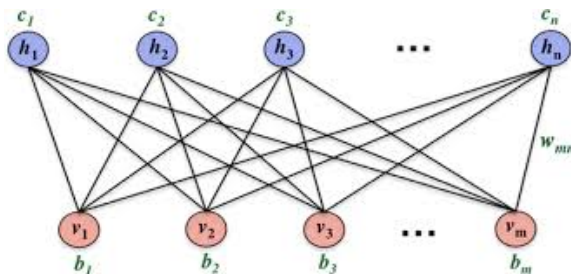


Figure 1: Graphical Representation of a RBM

2.1.2 Gibbs Sampling

Gibbs sampling is an iterative sampling procedure used to draw approximate samples from the distribution represented by an RBM. It belongs to the family of Markov Chain Monte Carlo Methods (MCMC). Sampling directly from the joint law $p(v, h)$ is difficult, but the conditional distributions $p(h | v)$ and $p(v | h)$ are easy to compute. The idea is therefore to alternate between them.

Starting from an initial visible vector $v^{(0)}$, one first samples a hidden vector according to

$$h^{(0)} \sim p(h | v^{(0)}),$$

then reconstructs a new visible vector from

$$v^{(1)} \sim p(v \mid h^{(0)}).$$

Repeating these two steps produces a Markov chain whose distribution progressively approaches the one defined by the RBM.

In practice, this procedure is used during training to approximate expectations that appear in the likelihood gradient. In particular, Contrastive Divergence relies on a short Gibbs chain, often only a few steps, to obtain an efficient approximation of the model statistics.

2.1.3 Contrastive Divergence 1 (CD-1)

Training an RBM exactly is intractable because the log-likelihood gradient depends on expectations that are too costly to compute. A common approximation is *Contrastive Divergence with one step* (CD-1).

Starting from a data sample $v^{(0)}$, CD-1 performs one Gibbs step:

$$h^{(0)} \sim p(h \mid v^{(0)}), \quad v^{(1)} \sim p(v \mid h^{(0)}).$$

Then the hidden probabilities are recomputed from the reconstruction:

$$p(h_j = 1 \mid v^{(1)}).$$

The idea is to move the model toward real data and away from its one-step reconstruction. Using this approximation, the gradients of the log-likelihood L are written as

$$\frac{\partial L}{\partial w_{ij}} \approx v_i^{(0)} p(h_j = 1 \mid v^{(0)}) - v_i^{(1)} p(h_j = 1 \mid v^{(1)}),$$

$$\frac{\partial L}{\partial a_i} \approx v_i^{(0)} - v_i^{(1)}, \quad \frac{\partial L}{\partial b_j} \approx p(h_j = 1 \mid v^{(0)}) - p(h_j = 1 \mid v^{(1)}).$$

Thus, CD-1 updates the parameters by comparing a true sample $v^{(0)}$ with its one-step reconstruction $v^{(1)}$.

2.2 Deep Belief Network (DBN)

2.2.1 DBN as a Stack of RBMs

A Deep Belief Network (DBN) can be viewed as a deep generative model obtained by stacking several Restricted Boltzmann Machines (RBMs). The main idea is that the hidden representation learned by one RBM is used as the input of the next one, allowing the network to learn increasingly abstract features. In this way, the first layers capture simple local patterns, while deeper layers model higher-level structures. From a probabilistic point of view, a DBN is not simply a deep RBM. The top two layers form an undirected bipartite model of RBM type, whereas the lower layers are connected in a directed top-down manner. Nevertheless, in practice, a DBN is conveniently constructed and trained as a stack of RBMs, which makes learning tractable and interpretable.

2.2.2 Greedy Layer-Wise Procedure

The standard training strategy for a DBN is the *greedy layer-wise procedure*. Instead of training the whole deep model at once, the layers are trained one after another. First, an RBM is trained on the original input data. Then, its hidden activations are computed and used as the training data for the next RBM. This process is repeated for all hidden layers.

Algorithmically, if $X^{(0)} = X$ denotes the original dataset, then for each layer l ,

$$\text{train RBM}_l \text{ on } X^{(l-1)}, \quad X^{(l)} = p\left(h^{(l)} = 1 \mid X^{(l-1)}\right).$$

This greedy pretraining allows each layer to learn useful features before passing them to the next one, and provides an efficient approximation to training the full DBN.

2.3 Deep Neural Networks

A Deep Neural Network (DNN) is a multilayer feed-forward architecture obtained by adding a supervised fine-tuning stage to the previously pretrained layers. While the DBN acts as a generative model to capture the underlying distribution of the data, the DNN adapts these features for a specific discriminative task, such as classification or regression.

In this context, the weights initialized by the greedy layer-wise RBM training serve as a starting point, placed into a standard MLP framework. A final output layer (e.g., softmax for classification) is appended to the stack. The entire network is then trained using backpropagation to minimize a cost function $\mathcal{C}$, such as the cross-entropy loss:

$$\mathcal{C}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik})$$

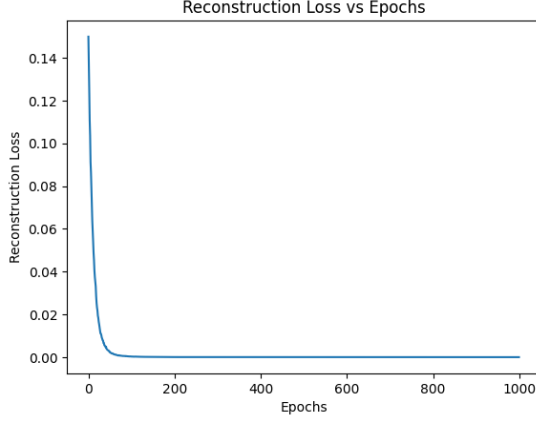
where y represents the true labels and $\hat{y}$ the model predictions. This transition from unsupervised pretraining to supervised fine-tuning allows the model to escape poor local optima and achieve better generalization than networks initialized with random weights.

3 Algorithmic Implementation and Results

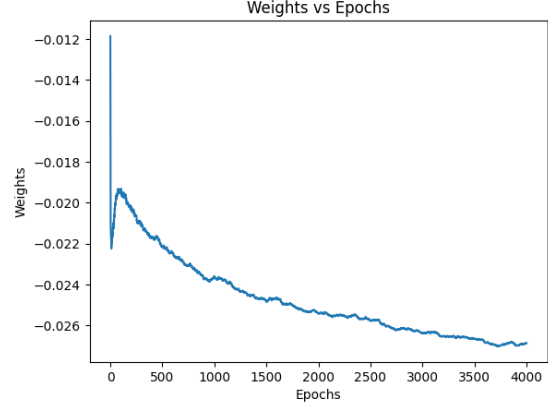
Following the project’s guidelines, we implemented all the algorithms (RBM, DBN, DNN) and benchmarked the RBM and DBN performance against the BinaryAlphaDigits data, and the DNN on the MNIST dataset.

3.1 Image Generation through RBM and DBN

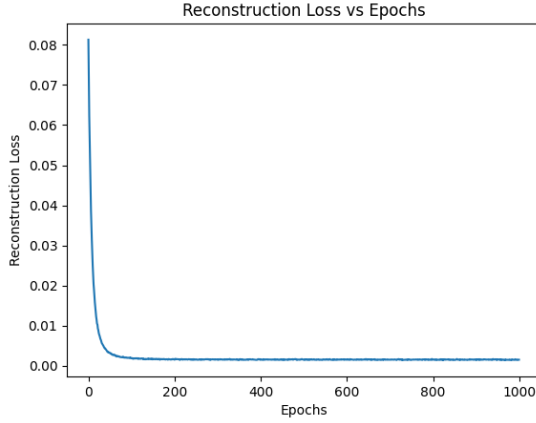
Here, we tested both RBM and DBN models for generating images through unsupervised learning on the Binary AlphaDigit dataset. We compared the quality of images generated based on the number of characters included in the dataset, the number of hidden units and number of hidden layers/ stacked RBMs for the DBN. The RBM is trained through the Contrastive Divergence 1 algorithm and the DBN is trained through the Greedy Layer Wise Procedure algorithm described before. We aimed to see the quality of generated images when we expand the number of characters in the dataset (thus making the probability distributions more complex) as well as when the models have more complex architectures. We will compare the models’ performances for similar datasets and architecture’s complexity. The experiments were conducted with several dataset and model configurations. For the datasets, we considered the character sets [3], [3, 8], [0, 1, 18, 3], and [0, 1, 2, . . . , 19]. For the RBM, we tested two hidden-layer sizes: 50 and 600 hidden units. For the DBN, we considered the architectures [320, 200] (one hidden layer of size 200), [320, 500, 200] (two hidden layers of sizes 500 and 200), and [320, 400, 500, 600, 800] (four hidden layers of sizes 400, 500, 600, and 800).



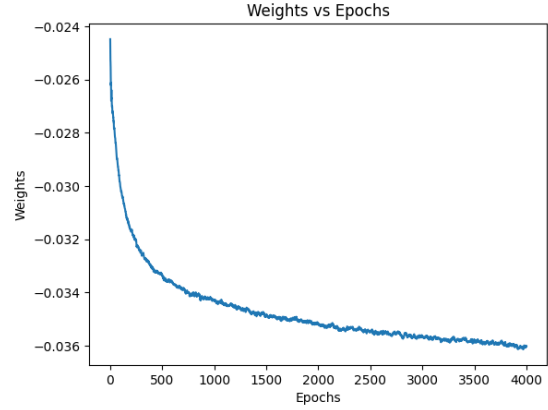
(a) RBM reconstruction loss



(b) RBM mean weight evolution



(c) DBN reconstruction loss



(d) DBN mean weight evolution

Figure 2: Training curves for the RBM and DBN models.

Figure 2 shows that both models learn quickly during the first epochs, as reflected by the sharp decrease in reconstruction loss. The RBM reaches a very small loss after a short training period, which indicates that it is able to reconstruct the training samples well. The DBN also exhibits a fast decrease, although its loss stabilizes slightly above zero, suggesting a more complex optimization process due to the deeper architecture.

The weight curves show a progressive stabilization during training for both models. After an initial transient phase, the average weights decrease and then evolve more slowly, which suggests that the parameters converge toward a stable configuration. Overall, these curves indicate that both the RBM and the DBN were successfully trained, with the RBM

appearing slightly easier to optimize while the DBN remains capable of learning meaningful hierarchical representations.

Now we will compare the performance of both models based on an increase in the model architecture's complexity as well as an increase in the dataset's diversity. Each of the following figures include the number of characters in the dataset as well as the architecture of the models.

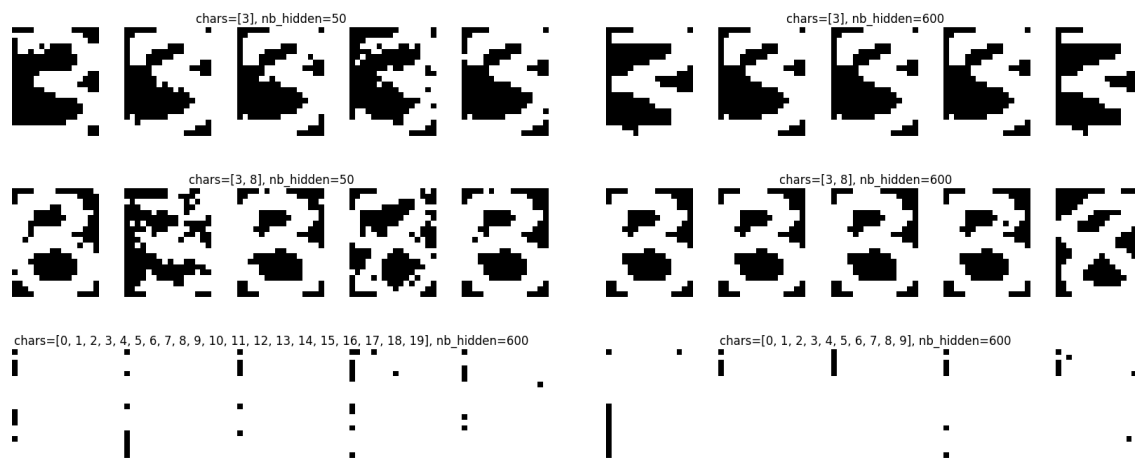


Figure 3: Generated samples obtained with the RBM for different numbers of characters in the training set and different hidden-layer sizes.

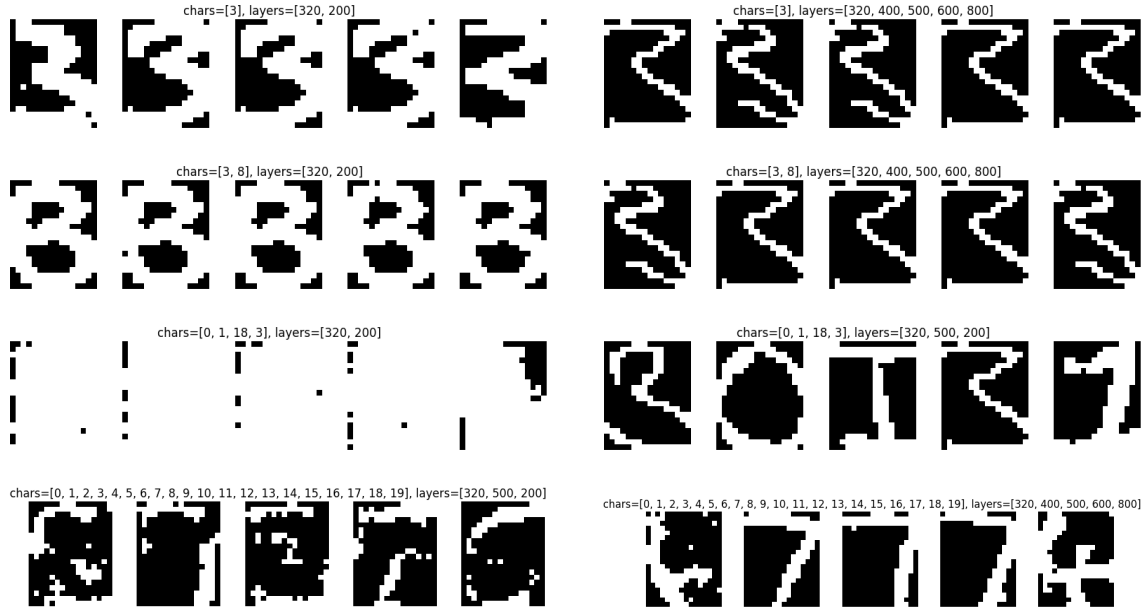


Figure 4: Generated samples obtained with the DBN for different numbers of characters in the training set and different architectures.

For the RBM experiments, we first considered the case where the training set contains only the character [3]. In this setting, the model was able to learn the distribution and generate realistic samples even without a very large number of hidden units. This can be explained by the low complexity of the dataset: since all training images correspond to the same character, the variability is limited, and even a relatively simple RBM is sufficient to capture the main structure of the data.

When the training set was enlarged to [3, 8], the task became more difficult. In this case, a higher number of hidden units was required to generate realistic images. The reason is that the model now has to represent two different visual patterns instead of one, which increases the diversity of the data distribution. With too few hidden units, the RBM does not have enough capacity to model both digits properly, and the generated images become less sharp or less coherent. With more hidden units, the model is better able to encode the variability of the two characters and produces more plausible samples.

For more complex datasets containing both numbers and letters, the RBM clearly reached its limits. Even the larger RBM was not able to generate convincing images when trained on a dataset containing 20 different characters. This shows that a shallow RBM is too simple to learn such a complex distribution. Indeed, the model has only one

hidden layer, which restricts its ability to capture the large diversity of shapes, strokes, and structures present in a dataset mixing many digits and letters.

For the DBN experiments, the results were better overall. For the datasets [3] and [3, 8], even the simpler DBN architectures were able to reproduce very realistic samples. This is expected, since the data distribution remains relatively simple. For [3, 8], however, we observed that the more complex DBN sometimes synthesized almost only the digit 3. A simple explanation is that the deeper model may have concentrated on the most stable or dominant mode of the distribution during training, instead of preserving a balanced representation of both digits. In other words, the additional complexity of the network did not necessarily improve diversity, and may even have pushed the model toward one preferred pattern.

For the dataset [0, 1, 18, 3], the simpler DBN failed to reproduce meaningful samples. This architecture was too limited to model a dataset with more diverse structures. In contrast, the more complex DBN was able to learn this richer distribution and produced samples resembling 0, 1, *I*, and 3. This is an important result, since in this setting the DBN succeeded where the RBM failed. It suggests that adding depth helps the model capture more abstract and hierarchical features, which becomes necessary when the dataset is more heterogeneous.

Finally, for the very complex dataset containing characters [0 to 20], the task became too difficult even for the DBN. One important point is that the number of training samples remains limited compared with the complexity of the distribution to be learned. Despite this difficulty, the more complex DBN was still able to reproduce a few recognizable structures, for example shapes similar to the digit 7. However, the overall visual quality remained poor, which confirms that this setting is too challenging for the considered models and training data. This experiment highlights the limitations of both RBMs and DBNs when the number of character classes becomes too large relative to the available data.

3.2 Comparison DNN : pre-trained vs random

First, we compared outputs from both pretrained and random initialized DNN. Due to computational bottleneck, many of the following experiments were ran with limited networks size and epochs. Specifically, we used :

- 15 RBM epochs
- 50 backpropagation epochs
- 24 batch size

- 10 000 train and 2 000 test data
- 0.05 learning rate
- 2 hidden layers of 200 neurons each

for the following analysis.

We initialized our 2 networks with the parameters described above in order to compare their performances. What we present here are the results for 8 sampled images, showing the true label and the predicted labels by both networks:

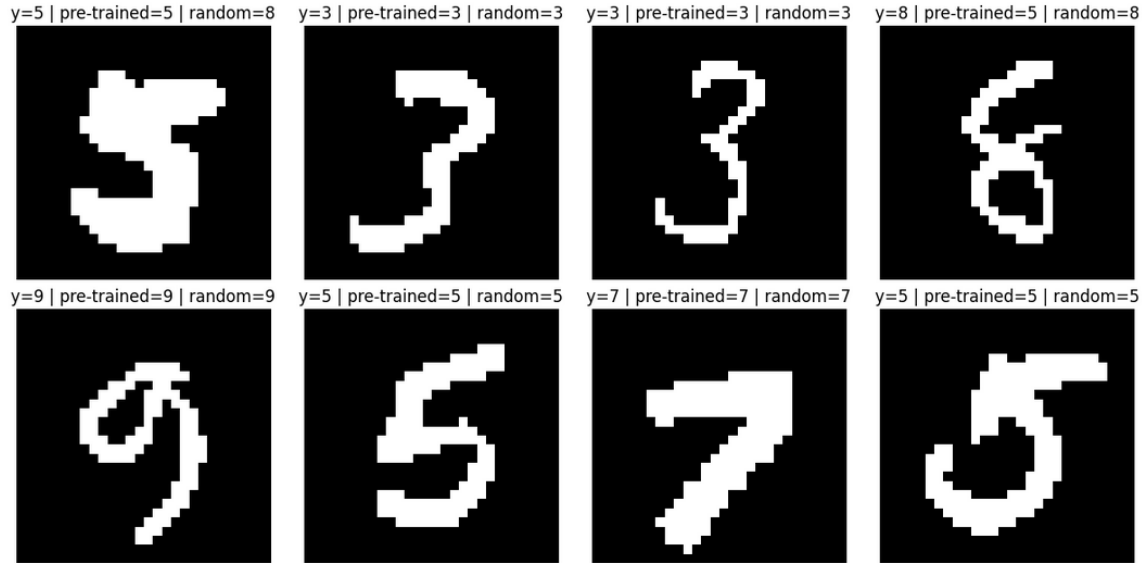


Figure 5: Sample for comparison between pre trained and random DNN

We observe that both networks have a sufficient performance on the clearly defined figures (like the 1, 7). However, it confuses 5 for 8 when the drawing is obviously bad. From those samples, we don't observe significant differences in the output. What is more interesting is to look into the output probabilities, which we show here as a heatmap :

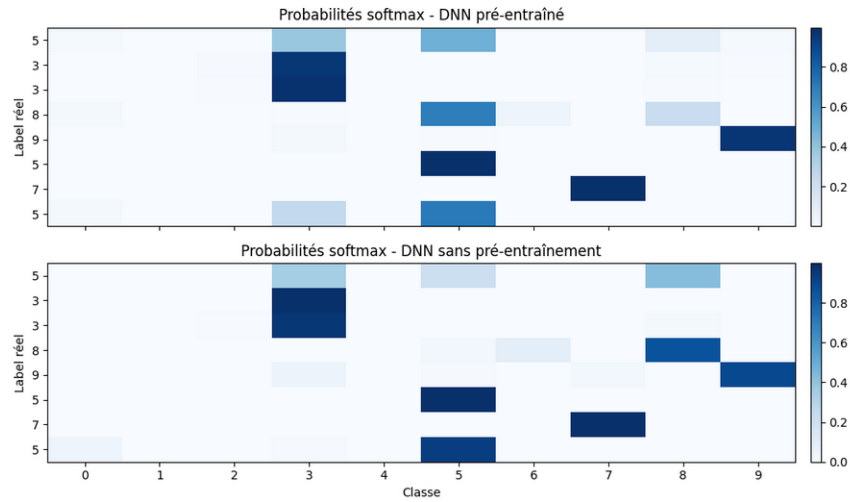


Figure 6: Heatmap probability for models output

Even if both networks have the same performance, some images show a slight difference. in the probability output, which suggest that the different initialization have had a little impact on the further recognition of the digits.

3.3 Comparison DNN : layers, neurons, data

Next, as advised in the project description, we ran both networks while varying 3 sets of parameters :

- The number of hidden layers : 1, 3, 5, 7
- The number of neurons per layer (using 2 layers) : 100, 300, 500, 700
- The number of training data (using 2 layers of 300 neurons) : 1000, 5000, 15000, 60000

Looking first into the layers :

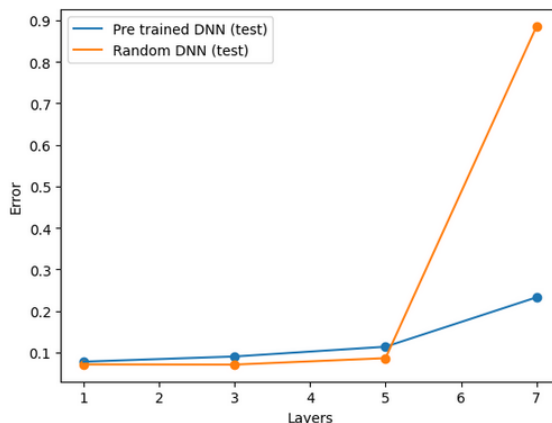


Figure 7: Evolution of the error depending on the number of layers

We observe an interesting phenomenon. Overall, the model seems to be overfitting with the number of layers. Second, the random DNN seems to be consistently better than the pre-trained one, apart from the 7 layers case where the networks completely overfits and does no better than random guessing on the test set. Apart from this, the performance is roughly equal between 1 and 5 layers (around 0.08 for both networks), with a slight increase tendency for the error. The best performance is achieved at 0.07 for the 3 layers random DNN. However, since we didn't use all the available data to compute the score, and the differences between the two being not that huge, the results may not be significant enough.

Indeed, when comparing our results with previous implementation, we saw a strong discrepancy between the usual outputs and our results. The error is expected to decrease with the number of layers (up until a certain point). Since we don't observe this phenomenon here, but the other plots being realistic, we believe it may be due to the way the data is sampled for the training, using only one sixth of the available data, and a random permutation. The results would need to be cross checked with larger training sizes.

Now for the neurons :

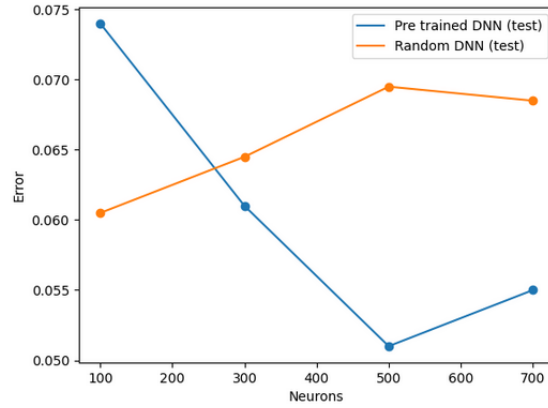


Figure 8: Evolution of the error depending on the number of neurons

The pattern is quite different depending on the network used. The error for pre trained DNN diminishes consistently until reaching 500 neurons, which seems to be the optimal points, and then goes up, certainly due to overfitting. At reverse, for the random DNN, the higher error rate is at 500 neurons, the less being the better in term of error.

Looking into the samples :

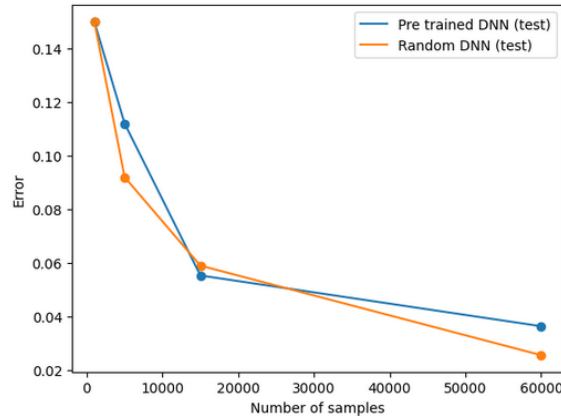


Figure 9: Evolution of the error depending on the number of training data

As expected, the error rate decreases exponentially with the number of samples used for the training. It is worth noting that all previous computations used only 10 000 samples ; we can infer what would have been the model performance with the full dataset, the error being roughly 2 times less when using all the data at our disposal. Both models scales similarly, and we believe the few differences between curve behavior are solely due

to stochasticity of the training.

4 Advanced Generative Models on MNIST

In this part, we test our generative models (RBM and DBN) on the MNIST dataset, and compare them to more advanced models such Variational Autoencoders (VAE), Generative Adversarial Networks (GANs), Diffusion Models (DDPM) and Score Based Generative Modeling (SGM). For GANs and VAEs, we will implement them from scratch using the class material, while for DDPM and SGM we will rely on two implemented architectures.

4.1 RBMs

Here we just trained our regular RBMs using the previous functions on the MNIST dataset, aiming to recover the probability distributions. It was trained for 200 epochs and 256 hidden units.

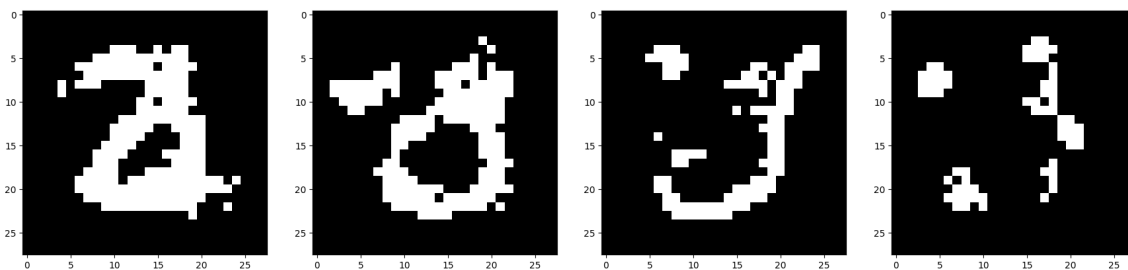


Figure 10: Digits generated with RBMs

We can clearly see that the model performs really bad. As explained before, this is a simple model that cannot recover a complex probability distribution.

4.2 DBN

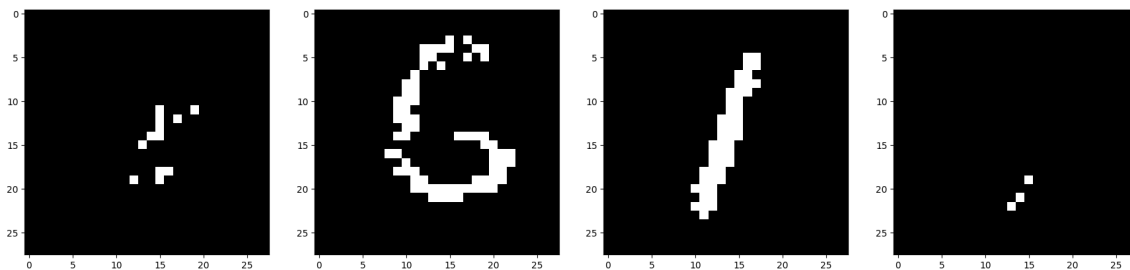


Figure 11: Digits generated with DBN

Compared with the RBM, the DBN produces slightly more structured samples. However, the generated images remain limited in diversity: the model mainly produces patterns resembling the digits 1 and 6, while the other digits are either poorly formed or almost absent

4.3 Variational Autoencoders (VAEs)

Variational Autoencoders (VAEs) are latent variable generative models that learn a probabilistic mapping between an observed sample x and a latent variable z . They are made up of an encoder $q_\phi(z | x)$, which approximates the posterior distribution of the latent variable, and a decoder $p_\theta(x | z)$, which reconstructs the data from the latent representation.

The model assumes the generative process

$$z \sim p(z), \quad x \sim p_\theta(x | z),$$

where $p(z)$ is usually chosen as a standard Gaussian distribution. VAEs optimize the Evidence Lower Bound (ELBO), defined as

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)).$$

Training consists in maximizing the ELBO, or equivalently minimizing its negative. The first term encourages accurate reconstruction of the data, while the second regularizes the latent distribution so that it remains close to the prior. In this sense, the ELBO approximates the maximum likelihood training of the generative model.

Thanks to the material provided by Prof. Alasdair Newson, we generated some MNIST samples using a VAE. Our first implementation uses a simple setting, with diagonal co-

variance matrix (that is null interaction term between latent dimensions) :



Figure 12: Digits generated with diagonal covariance matrix

Without much surprise, the results are not really good : the model doesn't represent the true underlying latent space. Hence, we tried with a (hopefully better) non diagonal gaussian model :



Figure 13: Digits generated with non-diagonal covariance matrix

We see a little improvement, but still far from the original MNIST samples. Finally, we used a true VAE to generate realistic figures :



Figure 14: Digits generated with VAE

We recognize some 9s and 8s with more precision. The advantage of a VAE compared to the other models that we did is that we are forcing some distributions (mainly the encoder one) to be close to a known prior. Still, the performances can be improved with better model and bigger network, but it shows promising steps.

4.4 Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) are generative models based on a two-player minimax game between a generator G and a discriminator D . The generator maps a latent random vector $z \sim p_z$ to a synthetic sample $G(z)$, while the discriminator outputs the probability that an input comes from the real data distribution rather than from the generator.

Their standard objective function is

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))].$$

During training, the discriminator is optimized to distinguish real samples from generated ones, while the generator is optimized to fool the discriminator by producing increasingly realistic data. At equilibrium, the generated distribution approaches the true data distribution, and the discriminator can no longer reliably separate real from fake samples.

To generate new data after training, one simply samples z from the latent distribution and computes $G(z)$.

We experimented a GAN architecture, trained to generate realistic images which would fool the discriminator. Starting from a white noise, here are our results after having trained the model for 400 epochs :

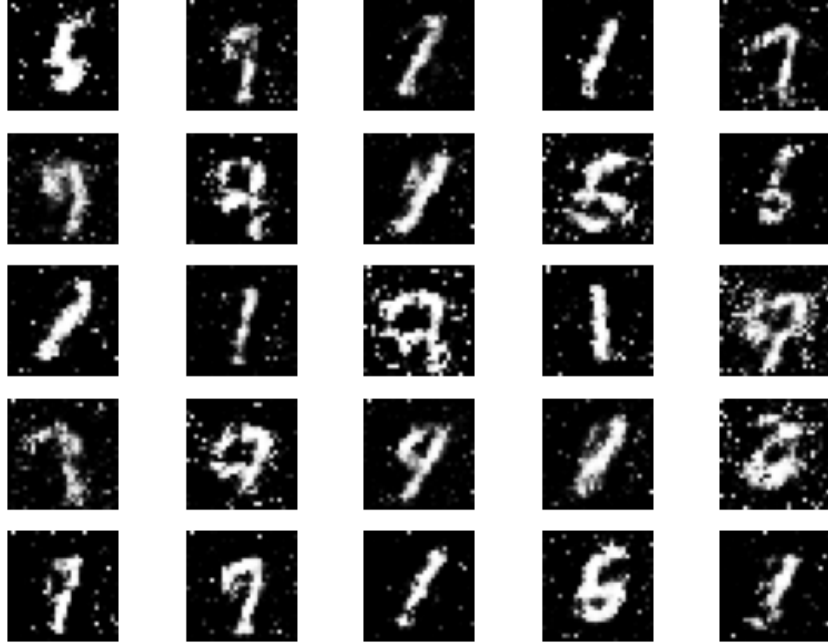


Figure 15: Digits generated with GAN

4.5 Denoising Diffusion Probabilistic Models (DDPM)

Denoising Diffusion Probabilistic Models (DDPMs) are generative models that learn to produce samples by reversing a gradual noising process. The forward process is defined as

a *Markov chain* that progressively corrupts a data sample x_0 into Gaussian noise:

$$q(x_t | x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I\right), \quad t = 1, \dots, T,$$

where $(\beta_t)_{t=1}^T$ is a variance schedule. After many steps, x_T becomes approximately standard Gaussian noise.

The model then learns the reverse Markov chain

$$p_\theta(x_{t-1} | x_t),$$

which aims to reconstruct the data distribution from noise. In practice, a neural network $\epsilon_\theta(x_t, t)$ is trained to predict the Gaussian noise added at step t . The standard training objective is

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2],$$

where

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

To generate a new image, one starts from pure noise $x_T \sim \mathcal{N}(0, I)$ and iteratively applies the learned reverse transitions until obtaining x_0 . This procedure can be viewed as a discrete-time stochastic sampling scheme related to *Langevin-type SDEs* and *Monte Carlo* generation, where samples are progressively refined from noise to structured data.

We trained a DDPM for 75 epochs on the MNIST dataset using the architecture proposed by <https://github.com/bot66/MNISTDiffusion> which relies on U-net and obtained the following results:

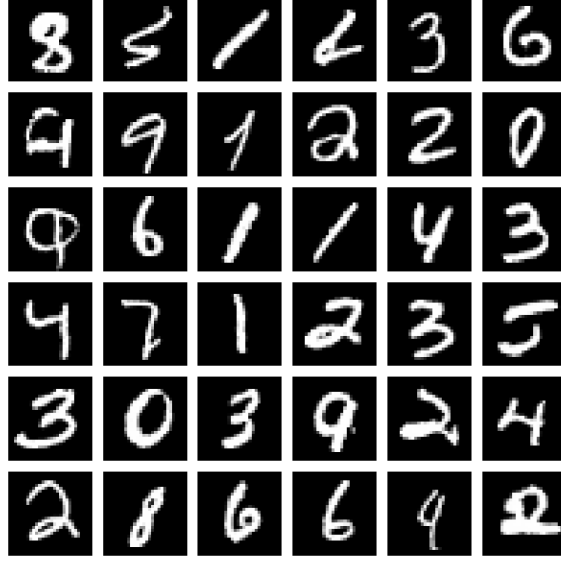


Figure 16: Digits generated with DDPM

We can clearly see that DDPM outperforms the four other generative models. The digits are very well represented and don't include any blurriness.

4.6 Score-Based Generative Modeling (SGM)

The final model tested is a score-based generative modeling technique based on the architecture proposed by <https://github.com/mfkasim1/score-based-tutorial>. The intuition behind it is the following:

A score-based model generates data by reversing a diffusion process that gradually adds Gaussian noise to an image $x_0 \in \mathbb{R}^{784}$. The forward process is defined by the SDE:

$$dx_t = -\frac{1}{2}\beta(t)x_t dt + \sqrt{\beta(t)} dW_t, \quad \beta(t) = 0.1 + (20 - 0.1)t.$$

Its marginal distribution is

$$x_t \mid x_0 \sim \mathcal{N}(\mu_t, \sigma_t^2 I),$$

where

$$\mu_t = x_0 \exp\left(-\frac{1}{2} \int_0^t \beta(s) ds\right), \quad \sigma_t^2 = 1 - \exp\left(-\int_0^t \beta(s) ds\right).$$

The network $s_\theta(x_t, t)$ is trained to approximate the score

$$\nabla_{x_t} \log p_t(x_t),$$

using the Gaussian conditional score

$$\nabla_{x_t} \log p(x_t | x_0) = -\frac{x_t - \mu_t}{\sigma_t^2}.$$

The loss is

$$\mathcal{L}(\theta) = \mathbb{E} \left[\lambda(t) \|s_\theta(x_t, t) - \nabla_{x_t} \log p(x_t | x_0)\|^2 \right].$$

To generate samples, the model starts from Gaussian noise and solves the reverse-time SDE

$$dx_t = \left[-\frac{1}{2}\beta(t)x_t - \beta(t)s_\theta(x_t, t) \right] dt + \sqrt{\beta(t)} d\bar{W}_t.$$

In practice, this process is discretized with Euler–Maruyama:

$$x_{t+\Delta t} = x_t + \left(-\frac{1}{2}\beta(t)x_t - \beta(t)s_\theta(x_t, t) \right) \Delta t + \sqrt{\beta(t)}\sqrt{|\Delta t|}z, \quad z \sim \mathcal{N}(0, I).$$

Starting from noise, the sample is progressively denoised into an MNIST digit.

After having run the model for around 60 epochs (we didn't have time to push the training further), we are left with those generated images:

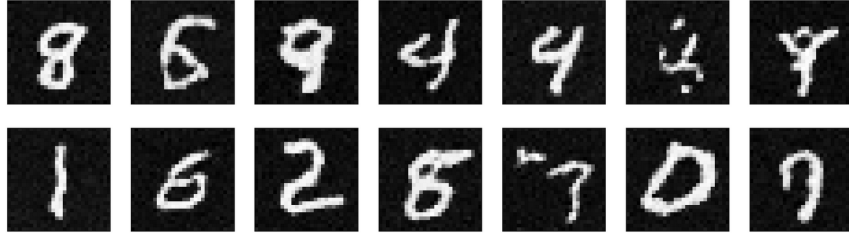


Figure 17: Digits generated with SGM

4.7 Comparison and Conclusion

In conclusion, the RBM remains a very simple generative model and was not expressive enough to capture the probability distribution of the MNIST dataset. The generated samples were fragmented and did not reproduce the global structure of handwritten digits. The DBN, thanks to its deeper architecture, achieved slightly better results and produced more

structured patterns, but it still failed to generate realistic and diverse samples. GANs and VAEs performed significantly better. GANs benefit from the adversarial training between the generator and the discriminator, which encourages sharper and more realistic images, while VAEs learn a structured latent space that allows stable generation and reconstruction. However, both models still produced some noisy or blurry samples, showing that their outputs were not always fully clean. Finally, DDPMs and score-based generative models (SGMs) gave the most realistic results, even without being trained for as many epochs as the previous models. This can be explained by their progressive denoising mechanism, which starts from random noise and gradually refines the sample through many small steps. This iterative generation process is generally more stable and better able to capture the data distribution, leading to clearer and more realistic handwritten digits.

5 Code

Our implementation can be found at <https://github.com/clemdst/deep-belief-networks>. We used 3 scripts (`RBM.py`, `DBN.py`, `DNN.py`) along with one other python script to load the data. Those scripts are encapsulated in the folder `src`. To generate the figures, we used a notebook (`DNN MNIST` for example) in which we call the different functions and classes from the files. We ran the notebook using the environments proposed in Google Colab in order to save training and setup time. Even though the training to generate the plot was quite long (approximately 3 hours for the MNIST). For the bonus part we separated them into one jupyter notebook for each generative model (GAN, VAE, RBM, DDPM, SGM).